Dependent type theory with $\mathsf{U} : \mathsf{U}$ and an impredicative universe $\mathsf{Prop} \subseteq \mathsf{U}$ of definitionally proof-irrelevant propositions: typing rules and main results

Formalisation in `MINPROP/` (Agda 2.9.0, `--without-K`)

21 October 2026

Abstract

We present a minimal dependent type theory with Π-types, a universe U with $\mathsf{U} : \mathsf{U}$, and a universe $\mathsf{Prop} : \mathsf{U}$ of propositions with $\mathsf{Prop} \subseteq \mathsf{U}$, closed under impredicative Π, and with definitional proof irrelevance. Using a domain-theoretic model and a logical relation, we prove that Π is injective and that typing is preserved by head reduction. Everything is formalised in Agda with no postulates, no holes and no termination or positivity pragmas.

1 Syntax

Terms are given in de Bruijn style (`Syntax/Raw.agda`):

$$M, N, A, B ::= x \mid \mathsf{U} \mid \mathsf{Prop} \mid \Pi(x : A)\, B \mid \lambda(x : A)\, M \mid M\, N.$$

Contexts are snoc-lists $\Gamma ::= \cdot \mid \Gamma, x{:}A$. The file `Syntax/Typing.agda` defines three mutually inductive judgements:

$$\Gamma\ \mathsf{ctx} \qquad \Gamma \vdash M : A \qquad \Gamma \vdash M = N : A$$

Head reduction (`Syntax/Reduction.agda`) is generated by

$$(\lambda(x : A)\, M)\, N \to_{\mathsf{h}} M[N/x] \qquad \frac{M \to_{\mathsf{h}} M'}{M\, N \to_{\mathsf{h}} M'\, N}$$

and $\to_{\mathsf{h}}^{*}$ denotes its reflexive–transitive closure. In particular U, Prop and Π-types are head normal.

2 Typing rules

Contexts

$$\frac{}{\cdot\ \mathsf{ctx}} \qquad \frac{\Gamma \vdash A : \mathsf{U}}{\Gamma, x{:}A\ \mathsf{ctx}}$$

Typing

$$\frac{\Gamma\ \mathsf{ctx} \qquad (x{:}A) \in \Gamma}{\Gamma \vdash x : A}\ \textsc{Var} \qquad \frac{\Gamma \vdash M : A \qquad \Gamma \vdash A = B : \mathsf{U} \qquad \Gamma \vdash B : \mathsf{U}}{\Gamma \vdash M : B}\ \textsc{Conv}$$

$$\frac{\Gamma\ \mathsf{ctx}}{\Gamma \vdash \mathsf{U} : \mathsf{U}}\ \textsc{U} \qquad \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U}}{\Gamma \vdash \Pi(x : A)\, B : \mathsf{U}}\ \textsc{Pi}$$

$$\frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma, x{:}A \vdash M : B}{\Gamma \vdash \lambda(x : A)\, M : \Pi(x : A)\, B}\ \textsc{Lam}$$

$$\frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M : \Pi(x : A)\, B \qquad \Gamma \vdash N : A}{\Gamma \vdash M\, N : B[N/x]}\ \textsc{App}$$

The universe of propositions.

$$\frac{\Gamma\ \mathsf{ctx}}{\Gamma \vdash \mathsf{Prop} : \mathsf{U}}\ \textsc{Prop} \qquad \frac{\Gamma \vdash A : \mathsf{Prop}}{\Gamma \vdash A : \mathsf{U}}\ \textsc{Prop-U} \qquad \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{Prop}}{\Gamma \vdash \Pi(x : A)\, B : \mathsf{Prop}}\ \textsc{Pi-Prop}$$

The domain A in Pi-Prop is any type, for instance Prop itself, so Prop is impredicative: $\Gamma \vdash \Pi(X : \mathsf{Prop})\, X : \mathsf{Prop}$. There is no rule $\mathsf{Prop} : \mathsf{Prop}$.

Conversion

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash M = M : A}\ \textsc{Refl} \qquad \frac{\Gamma \vdash M = N : A}{\Gamma \vdash N = M : A}\ \textsc{Sym} \qquad \frac{\Gamma \vdash M = N : A \qquad \Gamma \vdash N = P : A}{\Gamma \vdash M = P : A}\ \textsc{Trans}$$

$$\frac{\Gamma \vdash M = N : A \qquad \Gamma \vdash A = B : \mathsf{U} \qquad \Gamma \vdash B : \mathsf{U}}{\Gamma \vdash M = N : B}\ \textsc{Conv-Eq}$$

$$\frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma, x{:}A \vdash M : B \qquad \Gamma \vdash N : A}{\Gamma \vdash (\lambda(x : A)\, M)\, N = M[N/x] : B[N/x]}\ \textsc{Beta}$$

$$\frac{\begin{array}{c}\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma, x{:}A \vdash B' : \mathsf{U} \\ \Gamma \vdash A = A' : \mathsf{U} \qquad \Gamma, x{:}A \vdash B = B' : \mathsf{U}\end{array}}{\Gamma \vdash \Pi(x : A)\, B = \Pi(x : A')\, B' : \mathsf{U}}\ \textsc{Pi-Cong}$$

$$\frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash M\, x = N\, x : B \qquad \Gamma \vdash M : \Pi(x : A)\, B \qquad \Gamma \vdash N : \Pi(x : A)\, B}{\Gamma \vdash M = N : \Pi(x : A)\, B}\ \textsc{Funext}$$

$$\frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M = M' : \Pi(x : A)\, B \qquad \Gamma \vdash N : A}{\Gamma \vdash M\, N = M'\, N : B[N/x]}\ \textsc{App-Fun}$$

$$\frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M : \Pi(x : A)\, B \qquad \Gamma \vdash N = N' : A}{\Gamma \vdash M\, N = M\, N' : B[N/x]}\ \textsc{App-Arg}$$

In Funext, M and N are weakened in the premise.

Conversion rules for Prop**.**

$$\frac{\Gamma \vdash A : \mathsf{Prop} \qquad \Gamma \vdash M : A \qquad \Gamma \vdash N : A}{\Gamma \vdash M = N : A}\ \textsc{Proof-Irr} \qquad \frac{\Gamma \vdash M = N : \mathsf{Prop}}{\Gamma \vdash M = N : \mathsf{U}}\ \textsc{Prop-U-Eq}$$

$$\frac{\Gamma \vdash A : \mathsf{Prop} \qquad \Gamma \vdash B : \mathsf{Prop} \qquad \Gamma \vdash A = B : \mathsf{U}}{\Gamma \vdash A = B : \mathsf{Prop}}\ \textsc{U-Prop-Eq}$$

$$\frac{\begin{array}{c}\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{Prop} \qquad \Gamma, x{:}A \vdash B' : \mathsf{Prop} \\ \Gamma \vdash A = A' : \mathsf{U} \qquad \Gamma, x{:}A \vdash B = B' : \mathsf{Prop}\end{array}}{\Gamma \vdash \Pi(x : A)\, B = \Pi(x : A')\, B' : \mathsf{Prop}}\ \textsc{Pi-Prop-Cong}$$

Together, Prop-U-Eq and U-Prop-Eq say that two propositions are convertible as elements of Prop exactly when they are convertible as elements of U.

Remark. The system contains $\mathsf{U} : \mathsf{U}$ and so is logically inconsistent (Girard's paradox). The results below do not depend on consistency. They concern the equational theory: they show that Proof-Irr does not make the theory degenerate.

3 Main results

Theorem 1 (Π-injectivity; `PiInjectivity.piInjectivity`)**.** *If* $\Gamma \vdash \Pi(x : A)\, B = \Pi(x : A')\, B' : \mathsf{U}$*, then* $\Gamma \vdash A = A' : \mathsf{U}$ *and* $\Gamma, x{:}A \vdash B = B' : \mathsf{U}$*.*

This is derived from the more general statement:

Theorem 2 (`PiInjectivity.piConv`)**.** *If* $\Gamma \vdash T = \Pi(x : A')\, B' : \mathsf{U}$*, then there are* A, B *with* $T \to_{\mathsf{h}}^{*} \Pi(x : A)\, B$*,* $\Gamma \vdash A = A' : \mathsf{U}$ *and* $\Gamma, x{:}A \vdash B = B' : \mathsf{U}$*.*

Corollary 3. *If* $\Gamma \vdash \Pi(x : A)\, B = \Pi(x : A')\, B' : \mathsf{Prop}$*, then* $\Gamma \vdash A = A' : \mathsf{U}$ *and* $\Gamma, x{:}A \vdash B = B' : \mathsf{U}$ *(by* Prop-U-Eq *and Theorem 1).*

Theorem 4 (Subject reduction; `SubjectReduction.subject-red1`)**.** *If* $\Gamma \vdash M : A$ *and* $M \to_{\mathsf{h}} N$*, then* $\Gamma \vdash N : A$*.*

In the β case, a λ typed through Prop-U would need $\Gamma \vdash \mathsf{U} = \Pi(x : A)\, B : \mathsf{U}$. This is refuted by Theorem 2, because U is head normal.

Theorem 5 (No confusion; `NoConfusion`)**.** *In every context, none of the following holds:* $\Gamma \vdash \mathsf{U} = \mathsf{Prop} : \mathsf{U}$*,* $\Gamma \vdash \mathsf{U} = \Pi(x : A)\, B : \mathsf{U}$*,* $\Gamma \vdash \mathsf{Prop} = \Pi(x : A)\, B : \mathsf{U}$*.*

4 Outline of the proof

Domain model. Types and terms are interpreted by relations $M \Downarrow_{\rho} u$ to finite elements u of a domain (`Model/Eval.agda`). The finite elements are

$$u ::= \bot \mid \mathsf{U} \mid \mathsf{Prop} \mid \mathsf{Fun}(g) \mid \Pi(a, f),$$

where g, f are finite lists of steps. The elements carry an order $\leq$, a compatibility relation and a sup. Membership $u : a$ ("u is an element of the type a") is defined by structural recursion on a

stage index, which is why no termination pragma is needed. The codes of propositions are given by a structural, stage-free predicate:

$$\mathsf{IsProp}(\bot), \qquad \mathsf{IsProp}(\Pi(a,f)) \iff \text{every value of } f \text{ satisfies } \mathsf{IsProp},$$

and $u : \mathsf{Prop}$ is defined as $u : \mathsf{U} \wedge \mathsf{IsProp}(u)$. Thus $\mathsf{Prop} \subseteq \mathsf{U}$ holds semantically by projection.

Lemma 6 (Collapse; `Domain/Collapse.prop-collapse`)**.** *If $u : a$ and $\mathsf{IsProp}(a)$, then $u = \bot$.*

Proof sketch. By induction on the rank of a. If $a = \bot$, its only element is $\bot$. If $a = \Pi(b, f)$, an element is either $\bot$ or a coherent $\mathsf{Fun}(g)$ with g non-empty. The first step of g has a value lying in a Prop code of smaller rank, so that value is $\bot$ by induction. But coherence requires step values to be different from $\bot$, a contradiction. □

Soundness (`Model/Soundness.agda`, `Model/SoundnessProp.agda`). If $\Gamma \vdash M : A$, then every approximant of M lies below an element of an approximant of A. If $\Gamma \vdash M = N : A$, then M and N have the same approximants. The case Proof-Irr follows from the Collapse lemma: every approximant of a proof is $\bot$, and $\bot$ approximates every term.

Logical relation and adequacy (`Validity/`, `Adequacy/`). The relation is defined by stratification on rank:

- $\mathsf{Val}(M, A, u, a)$ and $\mathsf{EqVal}(M, N, A, u, a)$ for terms;
- $\mathsf{ValTy}(M, a)$ and $\mathsf{EqValTy}(M, N, a)$ for types.

Each clause bundles head reductions together with conversions. At Prop, $\mathsf{ValTy}(M, \mathsf{Prop})$ says $M \to_{\mathsf{h}}^{*} \mathsf{Prop}$ and $\Gamma \vdash M = \mathsf{Prop} : \mathsf{U}$, by analogy with U.

The fundamental theorem (`Adequacy/Value.agda`) proves validity of every derivation, by induction on the derivation. The Prop rules are handled in `Adequacy/Prop.agda`:

- Pi-Prop reuses the Π-into-U case after weakening the codomain premise by Prop-U.
- Prop-U uses soundness: an approximant of $A : \mathsf{Prop}$ is a Prop code.
- U-Prop-Eq: for a Prop code u, the relation at (u, U) gives the relation at (u, Prop), since they differ only in the component $\mathsf{ValTy}(\mathsf{U}, \mathsf{U})$ versus $\mathsf{ValTy}(\mathsf{Prop}, \mathsf{Prop})$.
- Proof-Irr uses the Collapse lemma: the relation at $u = \bot$ holds trivially.

Theorem 2 is obtained by applying adequacy to the given conversion, at the environment of all $\bot$s and the identity substitution, at the code $\Pi(\bot, \mathsf{nil})$.

Formalisation. The development consists of 58 Agda files. They are checked with Agda 2.9.0 and the option `--without-K`; most files also use `--exact-split`. It contains no postulates, no holes, and no `TERMINATING`, `NO_POSITIVITY_CHECK` or `REWRITE` pragmas. Checking the whole development takes about 15 seconds. The main files are:

`Syntax/Typing.agda`	the rules above
`Domain/MemStage.agda`, `Domain/PropCodes.agda`	membership, IsProp
`Domain/Collapse.agda`	Collapse lemma
`Model/Soundness.agda`, `Model/SoundnessProp.agda`	soundness
`Validity/Stratified.agda`	the logical relation
`Adequacy/Value.agda`, `Adequacy/Prop.agda`	fundamental theorem
`PiInjectivity.agda`, `SubjectReduction.agda`, `NoConfusion.agda`	main results